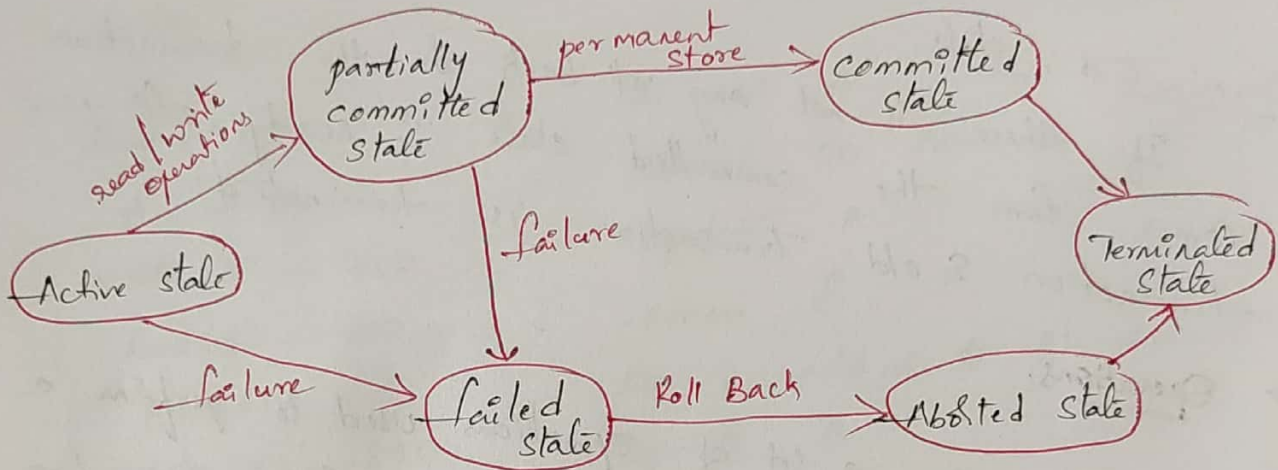


→ Transaction states & operations

A transaction goes during its life time is called a "state". This state decides the transaction whether it will commit or abort.



Active state:

Instructions of the transaction are running then the transaction is in active state. If all the read/write operations are performed without any error then it goes to partially committed state. If any instruction fails, it goes to the failed state.

partially committed:

After completion of all the read & write operations the changes are made in main memory. If the changes are made permanent on the database then the state will change to committed state & if it fails it will go to failed state.

failed state:

If any instruction of the transaction fails it goes to failed state.

Aborted state:

After having any type of failure the transaction goes from "failed state" to "Aborted state".

Committed state:

Transaction is completed then goes to committed state.

Terminated state:

If there is not any roll back of the transaction comes from the committed state & ready for new transaction & old transaction is terminated.

→ Operations:

Transactions are a set of operations used to perform a task. The transaction is any one execution. Executing the same program multiple times will generate multiple transactions.

Ex: ATM Transaction steps

1. Transaction start
2. Insert ATM card
3. Select language
4. Select Savings account option
5. Enter amount want to withdraw
6. Enter Secret pin
7. Wait for processing
8. Collect cash
9. Transaction completed.

Three operations can be performed

1. Read / Access data (R)
2. Write / change data (W)
3. commit

Ex: Transfer 50 from account A to B initially $A=500$
 $B=800$.

Read (A) --- 500	— Accessed A
$A = A - 50$	— deduct 50 from A
Write (A) --- 450	— update A
Read (B) --- 800	— Access B
$B = B + 50$	— 50 is added to B's Account
Write (B) --- 850	— update B
commit-	— complete transaction.

- All instructions before commit come under a partially committed state & stored in RAM.
- When the commit is read the data is fully accepted & is stored in Hard Disk.

→ Transaction properties

Transaction has four properties. These are used to maintain database before & after the transaction.

1. Atomicity
2. Consistency
3. Isolation
4. Durability

Atomicity:

It means either all successful or none. i.e. all states that all operations of the transaction take place at once if not then aborted.

Abort: All changes made are not visible

Commit: All changes of transaction made are visible

Consistency:

It means bringing the data base from one consistency state to another consistent state. The integrity constraints are maintained so that the data base is consistent before & after the transaction.

Isolation:

Transaction is isolated from other transactions. It shows the data which is used at the time of execution of a transaction can not be used by the second transaction until the first one is completed.

Durability:

It is used to indicate the performance of the database's consistent state. It states that the transaction made the permanent changes.

→ Requirement Condition: Schedules & Types of Schedules
A transaction must satisfy the ACID properties of DBMS.

- Atomicity
- Consistency
- Isolation
- Durability

- A schedule is defined as an execution sequence of transaction. A schedule maintains the order of the operation in each individual transaction. A schedule may contain a set of transactions.

Types:

2 types of Schedules

- Serial Schedule
- Concurrent Schedule

Serial Schedule:

All transactions are executed serially. After the instruction of T_i completes, the instructions of T_j will be executed, where $j = i + 1$.

For 2 transactions total no. of serial Schedules possible = 2.

$T_1 \rightarrow T_2$

$T_2 \rightarrow T_1$

For 3 transactions

possible = 6.

$T_1 \rightarrow T_2 \rightarrow T_3$

$T_2 \rightarrow T_1 \rightarrow T_3$

$T_3 \rightarrow T_1 \rightarrow T_2$

$T_1 \rightarrow T_3 \rightarrow T_2$

$T_2 \rightarrow T_3 \rightarrow T_1$

$T_3 \rightarrow T_2 \rightarrow T_1$

Concurrent Schedule:

multiple transactions can be executed in an interleaved manner but obeying the above ACID properties.

Ex:

$T_1 = 90 \text{ Sec}$

$T_2 = 500 \text{ Sec}$

$T_3 = 5 \text{ Sec}$

Serially :- $T_1 \rightarrow T_2 \rightarrow T_3$

then T_3 waits 590 Secs, so we go for non-serial (or) concurrent transactions to reduce waiting time.

$T_3 \rightarrow T_1 \rightarrow T_2$

→ Types of failures:

3 types of database transaction failures in DBMS.

1. System crash
2. Media failure
3. Application software errors.

System crash:

A system crash refers any kind of bugs (or) hardware malfunction in the operating system. It leads to loss of content residing on primary memory, cache memory, RAM etc. maintain strict security to reduce this.

Media failure:

It is caused by unreadable media. It leads to leave database system unavailable for several hours until recovery is complete. To prevent this we use malware protection & backing up our data frequently.

Application Software Errors:

resource limit is exceeded, bad inputs, logical or internal errors occur related to application software errors. to reduce these errors the developers put mechanisms and controls into place during the design of the architecture & coding operation.

→ System log

logs are used for recovering database from failure. it will have details about each step in the transaction so that in case of any failure data base can be recovered to the previous consistent state & taken to the consistent state after transaction.